Activation Service

Guia de uso da API pelo Postman: login OAuth2 e exemplos para os 3 endpoints

Endereço da API: `http://localhost:18080` Servidor de login: `http://localhost:18090`

A API registra pedidos de ativação de produtos. Cada pedido é aceito na hora com o status `PENDING` e, em segundo plano, o sistema chama o provedor de ativação e atualiza o status para `COMPLETED` (ativado) ou `FAILED` (não ativado).

1. Preparar o Postman (uma vez)

1. Instale o **Postman para desktop** (gratuito) em `postman.com/downloads`. Use o aplicativo instalado: a versão web do Postman não alcança endereços `localhost` sem um agente extra.
2. Abra o Postman. Criar conta é opcional; dá para seguir sem login em **Skip** ou **Continue without an account**.
3. Clique em **Import** (canto superior esquerdo) e selecione o arquivo da coleção, que acompanha o projeto:
   `docs/postman/activation-service.postman_collection.json`
4. A coleção **Activation Service** aparece na barra lateral, com 4 pastas e 16 requisições prontas.

> Cada requisição já tem o corpo preenchido e uma verificação automática do resultado esperado. Depois de enviar, a aba **Test Results** mostra em verde o que deu certo.

2. Login (OAuth 2.0)

O login fica configurado na própria coleção e vale para todas as requisições dela.

1. Na barra lateral, clique no nome da coleção **Activation Service**.
2. Abra a aba **Authorization**. O tipo já está como **OAuth 2.0**.
3. Desça até **Configure New Token** e confira os campos (já vêm preenchidos):

Campo	Valor
Grant type	`Client Credentials`
Access Token URL	`{{authUrl}}/default/token` (vira `http://localhost:18090/default/token`)
Client ID	`orders-system`
Client Secret	`secret`
Scope	`activation:read activation:write`
Client Authentication	`Send as Basic Auth header`

Neste ambiente de demonstração esses valores já funcionam. Num ambiente real, o **Client ID** e o **Client Secret** são enviados pela equipe responsável pela API.

4. Clique em `Get New Access Token` (no fim da tela).
5. Na janela **Authentication complete**, clique em `Proceed`.
6. Na janela **Manage Access Tokens**, clique em `Use Token`.
7. Pronto: o campo **Current Token** mostra o token, e todas as requisições da coleção passam a usá-lo (elas estão como **Inherit auth from parent**).

> **O token vale 15 minutos.** Se uma requisição começar a responder **401**, repita os passos 4 a 6: `Get New Access Token` → `Proceed` → `Use Token`.

Para executar qualquer requisição, o caminho é sempre o mesmo:

`clique na requisição` → `Send` → código e tempo no canto superior direito da resposta → corpo em **Body**, verificações em **Test Results**

3. POST /api/v1/requests — criar uma solicitação (pasta 1)

Campos obrigatórios: `externalId` (seu identificador do pedido), `customerId` e `productCode`, cada um com 1 a 64 caracteres: letras, números, `.`, `_` ou `-`.

O corpo das requisições usa `{{pedido}}`: a requisição 1.1 o preenche sozinha com um número novo (ex.: `PEDIDO-1793208843122`), então a sequência pode ser repetida quantas vezes quiser. O `id` devolvido também fica guardado para as consultas.

1.1 Criação válida

```
{
  "externalId": "{{pedido}}",
  "customerId": "CLIENTE-42",
  "productCode": "INTERNET-500"
}
```

Esperado: **201 Created**, `"status": "PENDING"` e o header `Location` (aba **Headers** da resposta).

1.2 Reenviar o mesmo corpo

Mesmo corpo da 1.1.

Esperado: **200 OK** com o **mesmo** `id` da 1.1. O sistema reconhece a repetição e não cria nada novo.

1.3 Mesmo externalId, outro produto

```
{
  "externalId": "{{pedido}}",
  "customerId": "CLIENTE-42",
  "productCode": "TV-100"
}
```

Esperado: **409 Conflict**: esse identificador já foi usado com outro conteúdo.

1.4 Dados inválidos

```
{
  "externalId": "PEDIDO 1002",
  "productCode": "INTERNET-500"
}
```

Esperado: **400 Bad Request**. A lista `errors` aponta `externalId` (tem espaço) e `customerId` (faltando).

1.5 Produto que o provedor recusa

```
{
  "externalId": "{{pedido}}-RECUSA",
  "customerId": "CLIENTE-42",
  "productCode": "INVALID-PRODUCT"
}
```

Esperado: **201** com `PENDING`. A recusa acontece depois; você a vê na requisição 2.2.

1.6 Provedor indisponível

```
{
  "externalId": "{{pedido}}-RETRY",
  "customerId": "CLIENTE-42",
  "productCode": "UNAVAILABLE-PRODUCT"
}
```

Esperado: **201** com `PENDING`. O sistema vai tentar de novo sozinho.

4. GET /api/v1/requests/{id} — consultar (pasta 2)

As requisições desta pasta já usam os `id` guardados pela pasta 1. Basta clicar em **Send**.

2.1 Ativação concluída

`GET {{baseUrl}}/api/v1/requests/{{requestId}}`

Esperado: **200** com `"status": "COMPLETED"`. A ativação ocorre em segundo plano em cerca de 1 segundo; esta requisição já espera 2 s antes de consultar.

2.2 Recusada pelo provedor

`GET {{baseUrl}}/api/v1/requests/{{failedRequestId}}`

Esperado: `"status": "FAILED"` e `"failureReason": "Provider rejected the request (HTTP 400)"`.

2.3 Tentativas automáticas

`GET {{baseUrl}}/api/v1/requests/{{retryRequestId}}`, logo após a 1.6 e de novo cerca de 30 segundos depois.

Esperado: primeiro `PENDING` (o sistema tenta de novo a cada ~5 s); depois `FAILED` com `"Gave up after 5 attempts..."`.

2.4 Solicitação inexistente

`GET {{baseUrl}}/api/v1/requests/00000000-0000-0000-0000-000000000000`

Esperado: **404 Not Found**.

5. GET /api/v1/requests — listar (pasta 3)

Os filtros ficam na aba **Params** da requisição. Todos são opcionais: `status`, `customerId`, `page` (começa em 0) e `size` (de 1 a 100).

Requisição	Params	Esperado
3.1 Todas	nenhum	**200**, das mais novas para as mais antigas; `totalElements` com o total
3.2 Só as que falharam	`status` = `FAILED`	Apenas solicitações com `FAILED`
3.3 Por cliente, paginado	`customerId` = `CLIENTE-42`, `page` = 0, `size` = 2	No máximo 2 itens, todos do `CLIENTE-42`; `totalPages` indica quantas páginas existem. Mude `page` para 1 para a próxima
3.4 Página grande demais	`size` = 500	**400** com `"must be between 1 and 100"`

6. Permissões: cliente só leitura (pasta 4)

A pasta 4 tem o próprio login, de um sistema chamado `reporting`, que só pode consultar.

1. Clique na pasta **4. Permissões (cliente só leitura)** → aba Authorization.
2. Get New Access Token → Proceed → Use Token.
3. **4.1 Tentar criar sem permissão** → Send. **Esperado: 403 Forbidden**.
4. **4.2 Ler solicitação de outro sistema** (o `id` criado na 1.1) → Send. **Esperado: 404**: um sistema não enxerga as solicitações criadas por outro.

7. Rodar tudo de uma vez (opcional)

1. Faça o login da coleção (seção 2) e o da pasta 4 (seção 6).
2. Clique nos **três pontinhos** ao lado de **Activation Service** → Run collection → Run Activation Service.
3. O Postman executa as 16 requisições em ordem e mostra o resultado de cada verificação. Todas devem aparecer como **PASS**.

8. Criar uma requisição do zero (sem a coleção)

1. New → HTTP.
2. Método `POST` e URL `http://localhost:18080/api/v1/requests`.
3. Aba **Authorization** → **Auth Type** = `OAuth 2.0`; preencha os campos da tabela da seção 2, trocando `{{authUrl}}` por `http://localhost:18090`; depois Get New Access Token → Proceed → Use Token.
4. Aba **Body** → `raw` → `JSON`, e cole um dos corpos da seção 3 (com um `externalId` escrito por você).
5. Send.

9. Referência rápida

Código	Significa	O que fazer
201	Solicitação criada	Acompanhar pelo GET (pasta 2)
200	Sucesso (ou reenvio reconhecido)	Nada
400	Dados inválidos	Ler a lista `errors`
401	Sem token ou token expirado	Get New Access Token → Proceed → Use Token
403	Token sem a permissão necessária	Conferir o campo Scope e pedir novo token
404	Solicitação não existe (para você)	Conferir o `id`
409	`externalId` já usado com outro conteúdo	Usar outro `externalId`

> **Encontrou algo diferente do esperado?** Anote o `traceId` que vem no corpo da resposta de erro. Ele permite à equipe técnica localizar a requisição exata nos registros do sistema.